

Paradigma Orientado a Objetos
Facultad de Ingeniería (FAIN)

Mauro Viotto, Modzelewski Jan.

Fecha: 25/10/2026

Docente: León Maria Angela

Actividad: TPO - Entrega: Etapa 1

1. Análisis del enunciado

Actores

- Administrador
- Vecino (titular del lote)
- Persona autorizada por el titular, o residente
- Guardia de seguridad
- Empleado de mantenimiento
- Sistema/Planificador: Se considera actor secundario para los eventos automáticos: escalado a 24 h, cierre por 5 días, aviso de cámara a 48 h y marca de moroso

Requerimientos Funcionales, Reglas de Negocio y Supuestos por Módulo
Requisitos funcionales

- RF-01: Registrar reclamo (validando titular/autorizado).
- RF-02: Asignar un único responsable.
- RF-03: Avanzar estado guardando historial y comentarios.
- RF-04: Reabrir (en plazo) o cerrar reclamo resuelto.
- RF-05: Escalar reclamos de prioridad alta no atendidos.

Reglas de negocio

- RN-01: Prioridades: Seguridad (Alta), Mantenimiento (Media), Convivencia/Admin (Baja).
- RN-02: El avance de estado es lineal (abierto → asignado → en curso → resuelto → cerrado).
- RN-03: Máximo 5 reclamos en curso por empleado.
- RN-04: Reapertura máxima: 5 días tras resolución (sino, cierre automático).
- RN-05: Prioridad alta sin atención >24h se escala.

Supuestos indispensables

- S-01: "No atendido" significa que pasaron 24h en estado abierto o asignado.
- S-02: Al reabrir un reclamo resuelto, vuelve al estado asignado (con el mismo empleado).
- S-03: El límite de 5 reclamos se valida exclusivamente al intentar pasarlo a en curso.

2. Expensas y pagos

Requisitos funcionales

- RF-06: Emitir expensa mensual por lote.
- RF-07: Registrar pagos (efectivo, transferencia, tarjeta).
- RF-08: Imputar pago y calcular mora.
- RF-09: Gestionar marcas de morosidad y notificaciones.
- RF-10: Registrar asientos de ajuste (corrección de errores).

Reglas de negocio

- RN-06: Expensa = presupuesto mensual × coeficiente de participación.
- RN-07: Vence el día 10. Interés: 3% mensual sobre saldo impago.
- RN-08: Imputación: primero a intereses, luego a capital (del período más antiguo al nuevo).
- RN-09: >3 períodos impagos = estado moroso. Se quita al regularizar.
- RN-10: Los pagos registrados no se eliminan.

Supuestos indispensables

- S-04: Coeficiente = superficie del lote / superficie total del barrio.
- S-05: "> 3 períodos impagos" equivale a tener 4 o más expensas vencidas sin cancelar.

3. Luminarias, cámaras y mantenimiento

Requisitos funcionales

- RF-11: Registrar equipos y su estado.
- RF-12: Registrar fallas y generar orden de mantenimiento.
- RF-13: Cerrar órdenes de mantenimiento.
- RF-14: Alertar cámara sin funcionar >48h.

Reglas de negocio

- RN-11: Estados: operativo, con falla, en reparación, fuera de servicio.
- RN-12: Cámara = Alta. Luminaria = Normal (Alta si fallan 4 o más en un mismo sector).
- RN-13: Cámara >48h sin funcionar se informa al admin.
- RN-14: Cierre obligatorio con: fecha, técnico y descripción.

Supuestos indispensables

- S-06: Para la regla de luminarias (>3), se cuentan las que estén "con falla" o "en reparación".

4. Ingreso de visitas y proveedores

Requisitos funcionales

- RF-15: Registrar autorizaciones.
- RF-16: Registrar ingreso validando autorización.
- RF-17: Registrar egreso.

Reglas de negocio

- RN-15: Sin autorización previa vigente no hay ingreso.
- RN-16: Proveedores admiten autorización recurrente (máx 30 días).
- RN-17: Prohibido registrar un ingreso si la persona ya ingresó y no registró egreso.

Supuestos indispensables

- S-07: La "franja horaria" valida el ingreso; el sistema no bloquea registrar un egreso tardío.

5. Reportes

Requisitos funcionales

- RF-18: Emitir reportes (reclamos pendientes, morosos, equipos fuera de servicio).

6. Requisitos Técnicos (No Funcionales)

- Para garantizar el funcionamiento adecuado del sistema, se establecen diversos requisitos técnicos indispensables. En primer lugar (RNF-01), los datos deben persistir en archivos locales entre las distintas ejecuciones de la aplicación. Asimismo (RNF-02), la interacción con el usuario se llevará a cabo a través de una interfaz por consola estructurada mediante menús explicativos. En términos de robustez (RNF-03), el sistema debe contemplar el manejo de excepciones propias diseñadas para capturar e informar de manera adecuada cualquier incumplimiento de las reglas de negocio. Por último (RNF-04), la lógica central de negocio deberá estar respaldada y validada mediante un conjunto de pruebas unitarias.

Supuestos indispensables

- S-08: No hay login con contraseña; al iniciar la consola, el usuario selecciona qué rol ocupará para adaptar el menú.
- S-09: Los procesos de tiempo (escalado 24h, 48h, mora y cierres) se controlan mediante una clase Reloj simulada para poder probarlos en las pruebas unitarias.

2. Diagrama de casos de uso

Actores y sus Casos de Uso principales:

- Vecino (Titular / Autorizado)
    - Registrar Reclamo
    - Reabrir Reclamo (<<extend>> de Consultar Reclamo)
    - Autorizar Ingreso (Visita/Proveedor)
    - Registrar Falla de Equipo

- Administrador
    - Asignar Reclamo
    - Emitir Expensas
    - Registrar Pago
    - Registrar Asiento de Ajuste (<<extend>> de Registrar Pago)
    - Generar Orden de Mantenimiento
    - Emitir Reportes

- Guardia de Seguridad
    - Registrar Ingreso de Visita (<<include>> Validar Autorización)
    - Registrar Egreso de Visita
    - Registrar Falla de Equipo

- Empleado de Mantenimiento
    - Actualizar Estado de Reclamo
    - Cerrar Orden de Mantenimiento

- Sistema / Planificador
    - Ejecutar Procesos Automáticos (Engloba escalados, morosidad, cierres automáticos y alertas por tiempo).

CU-01: Presentar Reclamo

- Este caso de uso es llevado a cabo por el Vecino, quien debe estar debidamente identificado como titular o residente dentro del sistema. Durante el flujo principal, el actor selecciona la opción "Nuevo Reclamo", especifica la categoría correspondiente (como seguridad o mantenimiento) y provee una descripción detallada del problema. A continuación, el sistema asigna la prioridad correspondiente, registra el reclamo bajo el estado "Abierto" y genera un número de identificación único. En cuanto a las extensiones y flujos alternativos, si el vecino consulta un reclamo que ya fue resuelto o cerrado y

han transcurrido menos de 5 días desde la resolución, el sistema habilita la posibilidad de "Reabrir", retornando el reclamo al estado "Asignado".

CU-02: Gestionar Reclamo

- En este caso de uso intervienen el Administrador y el Empleado de Mantenimiento, requiriendo como precondición la existencia de un reclamo en estado "Abierto". En el flujo principal, el Administrador asigna el reclamo a un empleado específico, quien posteriormente modifica su estado a "En curso" y finalmente a "Resuelto", incorporando obligatoriamente una observación en cada actualización. El sistema registra internamente la fecha y mantiene el historial del estado. En los flujos alternativos, si un empleado alcanza el límite de 5 reclamos activos al momento de asignarle o avanzar un caso, el sistema interrumpe la acción. Por otra parte, si un reclamo de prioridad Alta no recibe atención durante 24 horas, el sistema genera automáticamente una notificación dirigida al Administrador.

CU-03: Gestionar Expensas y Pagos

- Este caso de uso cuenta con la participación del Administrador y del Sistema como actor automático, teniendo como precondición contar con un presupuesto mensual previamente cargado. Durante el flujo principal, el Administrador emite las expensas mensuales, calculando el monto en función del presupuesto global y el coeficiente de participación correspondiente. Una vez emitidas, el Administrador efectúa el registro de los pagos, los cuales el sistema imputa prioritariamente a los intereses acumulados y luego al capital. Además, el día 11 de cada mes el sistema calcula de manera automática un 3% de interés por mora. Como flujo alternativo de extensión, si se registra un pago por error, el sistema no admite su eliminación directa y exige realizar un "Asiento de Ajuste" para regularizar el saldo.

CU-04: Reportar Falla de Equipo

- Llevado a cabo por el Vecino o el Guardia de Seguridad, este caso de uso requiere que el equipo involucrado (sea luminaria o cámara) se encuentre inicialmente en estado operativo. En el flujo principal, el usuario selecciona el dispositivo defectuoso y describe la falla presentada, tras lo cual el sistema actualiza el estado del equipo a "Con Falla" para su posterior revisión administrativa. En el flujo alternativo, si el equipo ya contaba con un reporte activo o había sido dado de baja ("Fuera de servicio"), el sistema notifica al usuario que no es posible generar un reporte duplicado.

CU-05: Gestionar Orden de Mantenimiento

- Este caso de uso involucra al Administrador y al Empleado de Mantenimiento, partiendo de la precondición de que existe un equipo registrado "Con Falla". En el flujo principal, el Administrador genera una orden de trabajo (asignando prioridad Alta si corresponde a cámaras o a más de 3 luminarias afectadas en un mismo sector). Posteriormente, el Empleado de Mantenimiento realiza la reparación y cierra la orden completando la fecha, su legajo y el detalle de los trabajos efectuados, lo que restablece el equipo al estado "Operativo". En el flujo alternativo, si el empleado intenta concretar el cierre de la orden sin brindar una descripción del arreglo, el sistema impide finalizar la operación hasta que el campo sea completado.

CU-06: Autorizar Ingreso

- Llevado a cabo por el Vecino bajo la precondición de estar registrado como titular o residente, este caso de uso permite autorizar el acceso a personas externas. En el flujo principal, el actor ingresa el nombre y DNI del visitante, indicando la fecha y la franja horaria permitida; con estos datos, el sistema incorpora el registro a la lista de autorizaciones vigentes. Como flujo alternativo, en caso de tratarse de un proveedor habitual, el usuario puede marcar la opción "Recurrente" para extender la validez de la autorización dentro de un rango de hasta 30 días consecutivos.

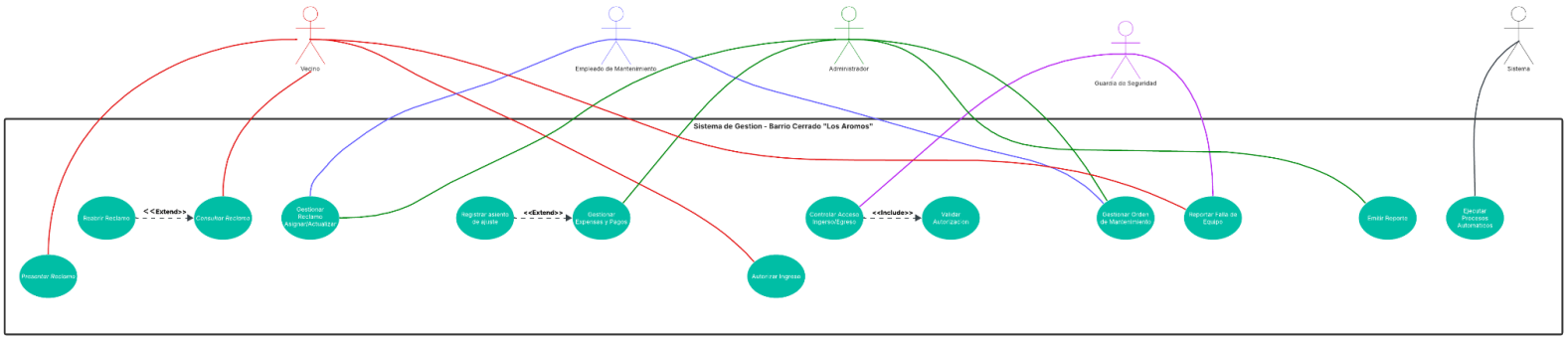
CU-07: Controlar Acceso

- Este caso de uso es ejecutado por el Guardia de Seguridad cuando una persona se presenta físicamente en la portería del barrio. En el flujo principal, el guardia ingresa el DNI del visitante y el sistema verifica mediante un proceso de inclusión que exista una autorización válida para la fecha y hora actuales. Al confirmarse la validez, el guardia registra el Ingreso y, posteriormente, marca el Egreso al momento de la salida. Dentro de los flujos alternativos, si la autorización es rechazada por el sistema, el guardia deniega el acceso; asimismo, si se intenta registrar el ingreso de una persona que registra una entrada previa sin su correspondiente marca de egreso, el sistema bloquea la operación.

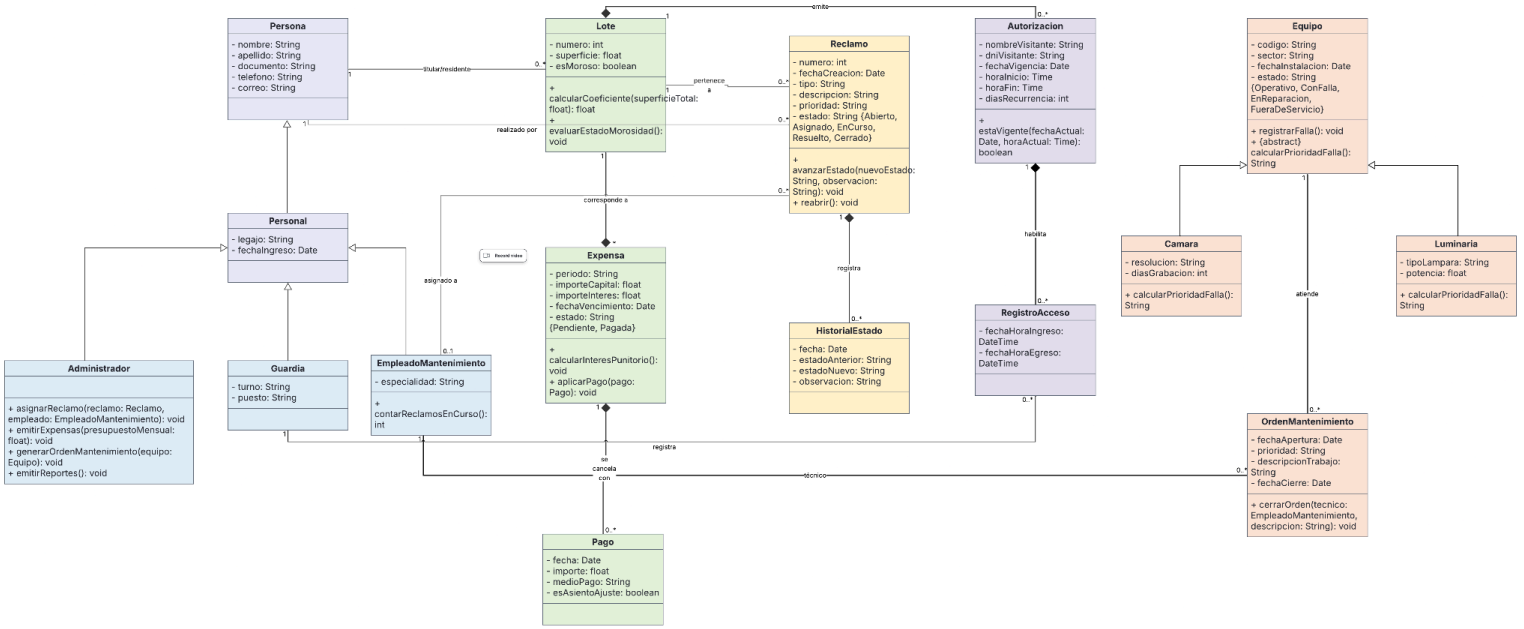
CU-08: Emitir Reportes

- Ejecutado por el Administrador bajo la precondición de contar con información almacenada en el sistema, este caso de uso permite consultar información consolidada. En el flujo principal, el actor selecciona la clase de reporte

deseado (tal como reclamos pendientes, lotes morosos o equipos fuera de servicio) y el sistema procesa los datos para presentarlos en pantalla. En el flujo alternativo, si no se detectan registros de morosidad o fallas en equipos para el criterio seleccionado, el reporte se genera indicando la leyenda "Sin novedades".



3. Diagrama de clases inicial

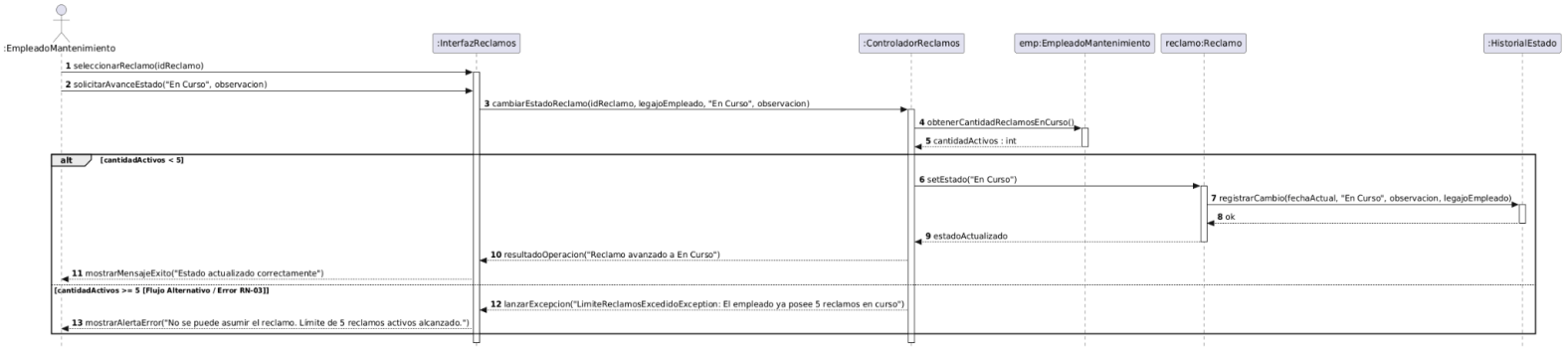


Este diagrama representa la primera aproximación estructural al dominio del sistema 'Los Aromos'. El diseño se centra en modelar fielmente las entidades del negocio y garantizar la integridad de los datos mediante relaciones de composición fuerte (como

Reclamo-HistorialEstado y Expensa-Pago). Como pilar fundamental de esta iteración, se implementaron jerarquías de herencia para el manejo del personal y se aplicó polimorfismo en la clase abstracta Equipo para resolver el cálculo de prioridades de mantenimiento de forma dinámica. En esta fase conceptual, las responsabilidades operativas se asignaron directamente a las clases que representan a los actores (como el Administrador), buscando una correspondencia directa y visual con las descripciones de los Casos de Uso, previo a la aplicación de patrones de desacoplamiento.

4. Diagramas de Secuencia

CU-02_GestionarReclamo



Este diagrama de secuencia modela el ciclo de vida operativo de un reclamo desde su asignación inicial hasta la resolución y registro histórico de cambios.

Flujo Principal: El Administrador interactúa con el sistema seleccionando un reclamo en estado "Abierto" y asignándole un EmpleadoMantenimiento. Posteriormente, el empleado actualiza el estado a "En curso" y "Resuelto". Ante cada cambio de estado, el objeto Reclamo instancia un nuevo objeto HistorialEstado para guardar la fecha, observación y nuevo estado, garantizando la trazabilidad.

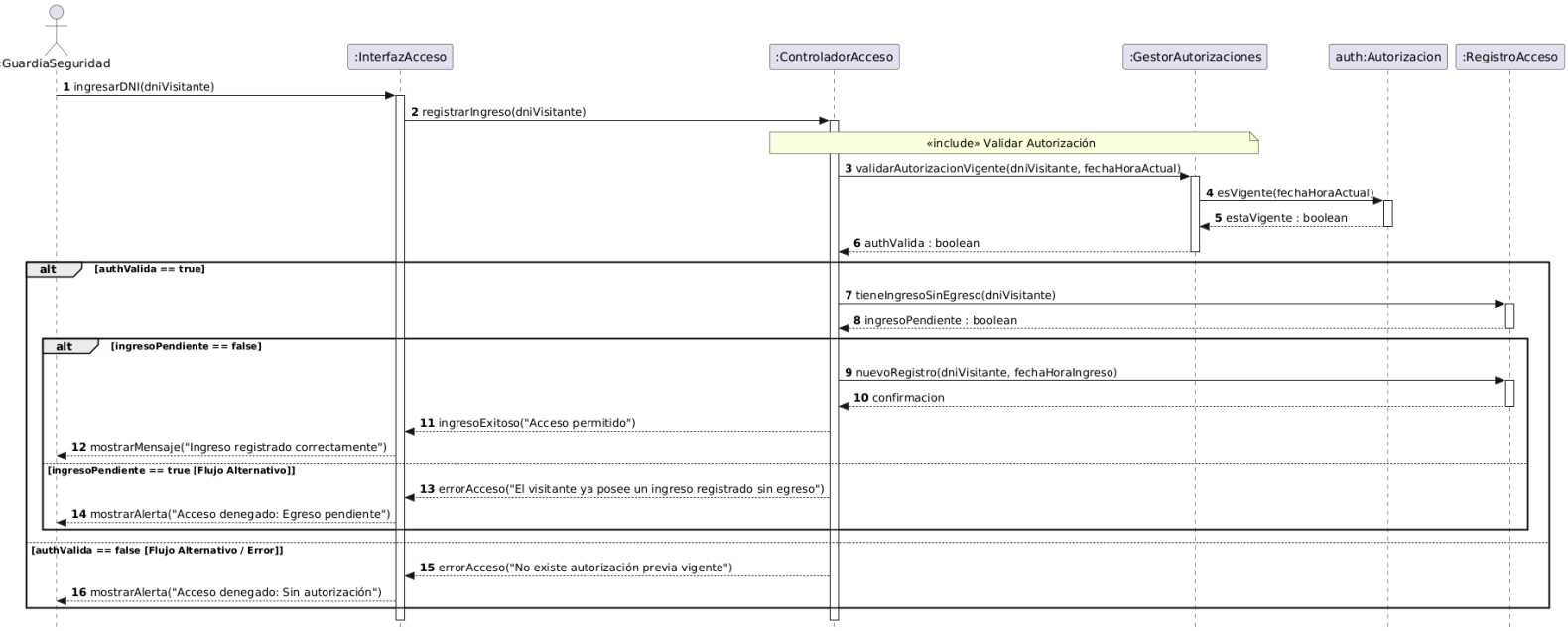
Flujos Alternativos y Fragmentos de Control:

- **Fragmento alt (Validación de carga laboral):** Antes de confirmar la asignación o el paso a "En curso", se invoca la verificación del límite de 5

reclamos activos (RN-03). Si el empleado supera dicha capacidad, el sistema interrumpe la operación y lanza una excepción notificando la restricción.

- **Fragmento opt (Escalado por tiempo):** Se representa la interacción del actor secundario Sistema (Reloj simulado), el cual verifica de manera periódica los reclamos de prioridad Alta. Si transcurren más de 24 horas sin atención (RN-05), se envía una notificación automática de escalado al Administrador.

CU-07_ControlarAcceso



Este diagrama representa la verificación y registro de visitas y proveedores que se presentan físicamente en la portería del barrio cerrado.

Flujo Principal: El Guardia ingresa el DNI del visitante en la interfaz. El ControladorAccesos consulta a la entidad Lote las autorizaciones registradas para validar si existe una Autorizacion vigente coincidente con la fecha y franja horaria actual (RN-15). Al confirmarse la validez, se crea la instancia de Ingreso asociando la fecha u hora de entrada y se retorna la habilitación al guardia. Posteriormente, al retirarse la persona, el guardia envía el mensaje para asentar la hora de Egreso.

Flujos Alternativos y Fragmentos de Control:

- **Fragmento alt (Sin autorización o fuera de rango)**: Si no existe una autorización cargada previamente por el vecino o el horario no coincide, el sistema retorna una denegación explícita de acceso.
- **Fragmento alt (Validación de estado / Doble ingreso):** Se evalúa la regla RN-17. Si la persona figura con un Ingreso activo al que aún no se le ha registrado su correspondiente Egreso, el sistema bloquea la operación e informa la inconsistencia de reingreso al guardia.

5. Aplicación de patrones

1. Principio de Responsabilidad Única (SRP - Single Responsibility Principle)

Problema detectado: Si la clase da la lógica de auditoría del seguimiento de estados (fechas de cambio y observaciones previas), la clase se volvería gigante y tendría múltiples motivos para cambiar.

Decisión tomada: Delegar la responsabilidad del registro histórico de cambios a una entidad dedicada y componerla dentro del reclamo.

Clases donde se refleja: Reclamo (gestiona la solicitud) e HistorialEstado (gestiona exclusivamente el registro de auditoría).

- **Ventaja:** Alta cohesión y facilidad de mantenimiento. Si en el futuro cambian las reglas de auditoría (ej. registrar la IP o el usuario), solo se modifica HistorialEstado sin afectar la lógica core del reclamo.

2. Principio Abierto/Cerrado (OCP - Open/Closed Principle)

Problema detectado: El barrio podría incorporar en el futuro nuevos dispositivos (por ejemplo, barreras de acceso o cercos eléctricos). Si la lógica dependiera de la clase

que genera la orden, habría que modificar el código base continuamente agregando condicionales if/switch.

Decisión tomada: Modelar una abstracción que permita extender el sistema creando nuevas clases derivadas, sin necesidad de modificar el código existente.

Clases donde se refleja: La clase abstracta Equipo está "abierta a la extensión" (pueden sumarse nuevos tipos de equipos) pero "cerrada a la modificación" (el método generarOrdenMantenimiento funcionará sin tocar su código original).

- **Ventaja:** Hace que el módulo de mantenimiento sea altamente escalable y a prueba de cambios a futuro.

3. Principio de Inversión de Dependencias (DIP - Dependency Inversion Principle)

Problema detectado: Los módulos de alto nivel (como el Administrador al generar una orden de trabajo) no deberían depender de detalles concretos de bajo nivel (como saber si el aparato roto es específicamente una cámara 4K o una lámpara LED).

Decisión tomada: Hacer que la generación de la orden de mantenimiento dependa de una abstracción general y no de la implementación específica del aparato.

Clases donde se refleja: El método generarOrdenMantenimiento(equipo: Equipo) del Administrador recibe la clase abstracta Equipo, invirtiendo la dependencia.

- **Ventaja:** Reduce drásticamente el acoplamiento. La administración puede enviar a reparar cualquier cosa siempre y cuando herede de "Equipo", sin conocer sus detalles técnicos.

Patrones GRASP Aplicados

1. Polimorfismo

Problema detectado: Las cámaras generan un nivel de prioridad de reparación diferente (Alta) al de las luminarias (Normal o Alta si son más de 3). Controlar esto

desde un agente externo preguntando if (tipo == "Camara") rompe el encapsulamiento.

Decisión tomada: Usar operaciones polimórficas. Cada tipo de equipo sabe cómo calcular su propia prioridad según sus reglas de negocio.

Clases donde se refleja: El método abstracto calcularPrioridadFalla() en Equipo, que es sobrescrito con diferente comportamiento en las subclases Camara y Luminaria.

- **Ventaja:** Elimina el uso de lógica condicional rígida y asigna el comportamiento directamente al tipo responsable.

2. Experto en Información (Information Expert)

Problema detectado: ¿Qué clase del sistema debería ser la responsable de aplicar la regla de negocio que calcula el 3% de interés punitorio por mora?

Decisión tomada: Asignar la responsabilidad a la clase que posee la información necesaria (el importe base, la fecha de vencimiento y el estado) para realizar ese cálculo sin pedir datos prestados.

Clases donde se refleja: Expensa (a través de su método calcularInteresPunitorio()).

- **Ventaja:** Evita crear clases "Gestoras" innecesarias, manteniendo la lógica cerca de los datos que manipula (alta cohesión).

3. Creador (Creator)

Problema detectado: ¿Qué parte del sistema debería ser responsable de instanciar (hacer el new) los objetos HistorialEstado cada vez que un reclamo avanza de fase?

Decisión tomada: Asignar la responsabilidad de creación a la clase que agrega/compone fuertemente al objeto y que posee los datos de inicialización.

Clases donde se refleja: El método avanzarEstado() dentro de Reclamo es el que instancia internamente los nuevos HistorialEstado.

- **Ventaja:** Favorece el bajo acoplamiento, ya que evita depender de clases "Factory" externas para objetos que solo tienen sentido de existencia dentro de un reclamo.

4. Controlador (Controller) *(Punto clave para rediseñar el diagrama final)*

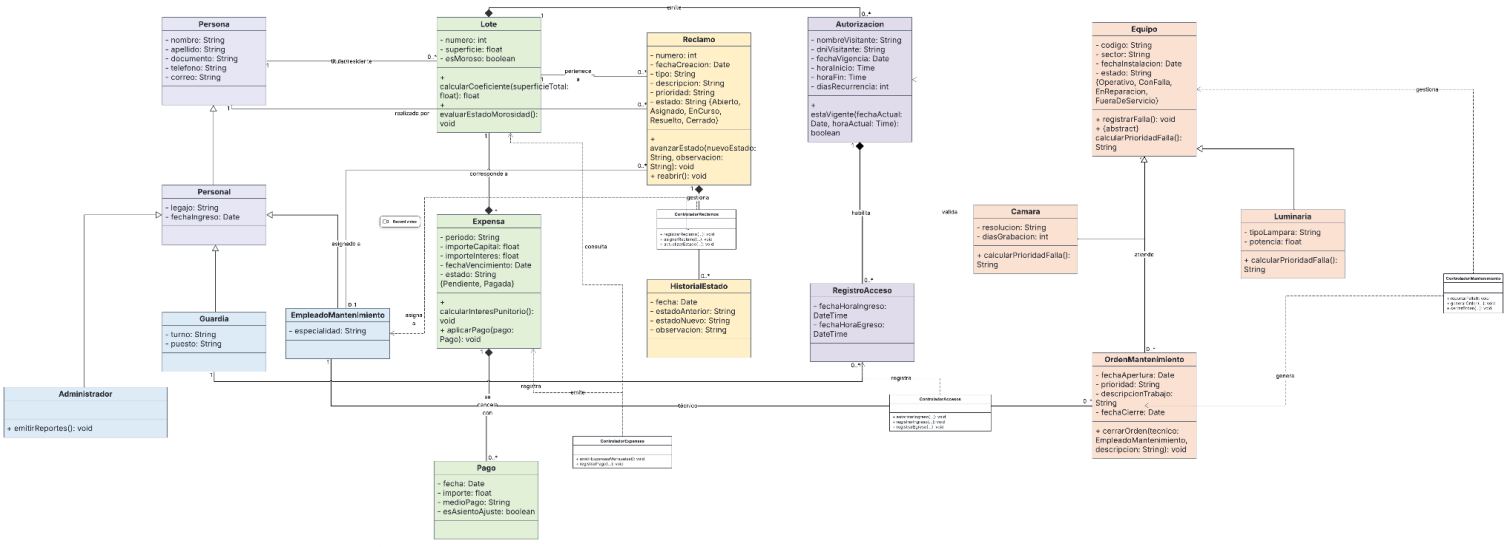
Problema detectado: En el diagrama inicial, los actores (como el Administrador o el Vecino desde la consola gráfica) interactúan directamente con las clases del dominio (Lote, Reclamo, etc.). Si la interfaz (menú de consola) cambia, se expone innecesariamente el dominio.

Decisión tomada: Interponer objetos controladores (manejadores de casos de uso) que reciban los eventos del usuario desde la interfaz gráfica o menú, y deleguen el trabajo a los objetos internos del sistema.

Clases donde se refleja (Nuevas para el diagrama final): Creación de clases ControladorReclamos, ControladorExpensas y ControladorAccesos.

- **Ventaja:** Separa limpiamente la capa de presentación de la capa de negocio, protegiendo las clases estructurales de las modificaciones en la interacción con el usuario.

6. Diagrama de clases final



Respecto de la versión inicial, se rediseñó el diagrama incorporando clases Controladoras (patrón GRASP). Esto extrae la lógica de orquestación que estaba acoplada a las clases de los actores (como el Administrador) y la centraliza en manejadores de casos de uso. De esta forma, protegemos el dominio, bajamos el acoplamiento y preparamos el sistema para la interfaz por consola solicitada en los requisitos no funcionales.

Este rediseño alinea la estructura estática con los casos de uso y diagramas de secuencia elaborados. Las clases controladoras (ControladorReclamos, ControladorAccesos y ControladorExpensas) asumen la ejecución de los flujos principales y alternativos, incorporando en el diagrama final los métodos y firmas necesarios para dar soporte a las interacciones dinámicas. De este modo, operaciones como la validación de carga laboral de los empleados, el control de franjas horarias de autorización, el escalado automático de reclamos y la instanciación del historial de estados quedan reflejadas en el modelo de clases, garantizando trazabilidad y coherencia en el diseño.